

Proposed LangGraph Design

1. Goal

When a Jenkins failure webhook is received, the application should pass the captured failure log into a LangGraph workflow that:

1. extracts all vulnerability issues from the failure output
2. fetches repo manifests and Docker files
3. generates BOM-friendly remediation options
4. validates that the chosen changes are BOM-safe and operationally safe
5. pushes approved changes to a new branch

2. Keep vs add

Keep from the current codebase:

- `POST /api/v1/webhooks/jenkins/failure`
- event persistence concept
- current repo/branch/build metadata fields
- GitHub file fetch capability
- GitHub branch and PR integration concepts

Add:

- a LangGraph state model
- four graph nodes
- a structured vulnerability parser for Trivy-like console logs
- manifest parsers for Maven, Gradle, and Dockerfile
- a multi-file change plan and multi-file GitHub commit path
- explicit validation output before push

3. Proposed runtime architecture

3.1 Ingress

The webhook endpoint should:

1. receive payload
2. normalize and store the raw console text
3. persist a run record with event metadata
4. enqueue or background-trigger a LangGraph invocation using the stored event id
5. return 202 Accepted

This keeps Jenkins integration stable while allowing slower AI processing outside the request-response critical path.

3.2 LangGraph entry state

Suggested graph input:

```
{
  "event_id": "evt_xxx",
  "repo": "owner/repo",
  "branch": "main",
  "build_url": "...",
  "local_repo_path": "... or None",
  "console_text": "...raw Jenkins failure output...",
  "requested_base_branch": "main",
  "requested_mode": "auto-remediate",
}
```

4. Proposed graph nodes

Node 01: Vulnerability extraction and general solutioning

Responsibilities:

- parse the Jenkins failure log

- detect whether the failure contains vulnerability analysis output
- extract all findings, not just one
- produce a normalized issue list
- provide general remediation guidance per issue group

Primary outputs:

- `va_detected`: bool
- `issue_list`: list[VAFinding]
- `issue_summary`
- `general_solutions`
- `scan_targets` such as `pom.xml`, `build.gradle`, `Dockerfile`

Expected logic:

- support Trivy table-like output first
- group repeated CVEs under the same package when appropriate
- preserve all listed fixed versions
- capture severity, installed version, title, and target file

Why it is needed:

- the current `_analyze_event()` is not enough for table-form multi-finding scans
- the notebook work already points toward richer issue extraction and BOM-aware reasoning

Node 02: Repository file fetch and BOM-friendly remediation planning

Responsibilities:

- fetch relevant repo files using repo + branch
- inspect Maven, Gradle, and Docker artifacts
- determine whether a direct version bump, property bump, BOM upgrade, parent upgrade, or base image upgrade is the correct remediation type
- propose concrete candidate changes

Files to inspect:

- `pom.xml`
- `build.gradle`

- `build.gradle.kts`
- `gradle.properties`
- `settings.gradle`
- `settings.gradle.kts`
- `Dockerfile`
- nested module manifests when present

Primary outputs:

- `repo_files`
- `manifest_context`
- `remediation_candidates`
- `selected_change_plan_draft`

Important design rule:

- prefer a BOM or parent-driven fix before inline dependency version insertion
- prefer one higher-level upgrade that resolves many vulnerabilities over many leaf-level edits

Node 03: BOM-safety and change validation

Responsibilities:

- validate whether proposed fixes are consistent with Maven/Gradle dependency management
- reject unsafe changes
- refine or downgrade plans that violate BOM guidance
- ensure the graph only pushes changes after validation succeeds

Validation checks:

- if a dependency is managed by a Maven parent or imported BOM, do not add an inline version unless explicitly justified
- if a Gradle dependency is controlled by platform or version catalog, modify the control point rather than the leaf dependency
- if several CVEs can be resolved by upgrading one parent/BOM/version property, prefer that path
- if the fixed version list spans incompatible major versions, choose a policy-compliant target and mark confidence
- if the plan needs multiple files, confirm the full write set is complete before proceeding

Primary outputs:

- `validated_change_plan`
- `validation_passed`
- `validation_notes`
- `manual_review_required`

Node 04: Branch creation and GitHub push

Responsibilities:

- generate branch name
- write the approved file changes
- create or reuse remediation branch
- push the changes
- open PR or compare URL result
- persist run result back to the event/run record

Primary outputs:

- `branch_name`
- `changed_files`
- `commit_result`
- `pull_request_url`
- `final_status`

Critical gap vs current code:

- this node should support multi-file commits
- the current helper only updates one file at a time

5. Recommended state model

Suggested core types:

```
from typing import Literal
from pydantic import BaseModel
```

```
class VAFinding(BaseModel):
    target_file: str
```

```
package_name: str
severity: str
cve_id: str
installed_version: str | None = None
fixed_versions: list[str] = []
title: str | None = None
ecosystem: Literal["maven", "gradle", "docker", "unknown"] =
    "unknown"
```

```
class RemediationCandidate(BaseModel):
    candidate_id: str
    file_path: str
    change_type: Literal[
        "maven-parent-upgrade",
        "maven-bom-upgrade",
        "maven-property-upgrade",
        "maven-direct-dependency-upgrade",
        "gradle-platform-upgrade",
        "gradle-property-upgrade",
        "gradle-direct-dependency-upgrade",
        "docker-base-image-upgrade",
        "docker-package-upgrade",
    ]
    rationale: str
    proposed_target_version: str | None = None
    confidence: float
```

```
class GraphState(BaseModel):
    event_id: str
    repo: str | None = None
    branch: str | None = None
    build_url: str | None = None
    local_repo_path: str | None = None
    console_text: str
    va_detected: bool = False
    issue_list: list[VAFinding] = []
    general_solutions: list[str] = []
    repo_files: dict[str, str] = {}
    remediation_candidates: list[RemediationCandidate] = []
```

```
validated_change_plan: dict = {}
validation_passed: bool = False
manual_review_required: bool = False
branch_name: str | None = None
pull_request_url: str | None = None
final_status: str = "received"
```

6. Service boundaries

Recommended module split:

- `api/jenkins_webhook.py`
- `graph/state.py`
- `graph/builder.py`
- `graph/nodes/node01_extract_issues.py`
- `graph/nodes/node02_fetch_and_plan.py`
- `graph/nodes/node03_validate_bom.py`
- `graph/nodes/node04_push_branch.py`
- `services/repo_reader.py`
- `services/manifest_parser.py`
- `services/remediation_planner.py`
- `services/validation_service.py`
- `services/github_writer.py`

This keeps node files focused and prevents `jenkins_service.py` from becoming even more overloaded.

7. Integration with existing code

Recommended reuse map:

- reuse webhook route shape and event metadata fields from `jenkins_service.py`
- reuse `fetch_analysis_files()` patterns from `github_service.py`
- reuse branch naming ideas from `_build_branch_name()` in `jenkins_service.py`
- replace `_analyze_event()` with Node 01 output
- replace `_prepare_fix()` with Node 02 plus Node 03 outputs
- replace single-file `_apply_fix_to_github()` with a new multi-file GitHub writer used by Node 04

8. Execution model

Recommended control flow:

- Webhook request stores event.
- A background task invokes the graph with `event_id`.
- Graph execution updates a run record after each node.
- UI can poll the run status separately from the webhook call.

This is better than blocking the webhook while the graph fetches repo contents and waits on model calls.

9. Error handling model

Suggested node-level error classes:

- `PayloadParseError`
- `RepoFetchError`
- `ManifestParseError`
- `ValidationError`
- `PushError`

Suggested state fields:

- `errors: list[dict]`
- `retryable: bool`
- `last_completed_node: str | None`

10. Success criteria

The design is successful when:

- one Jenkins failure payload can produce a full issue inventory
- Trivy table output is parsed into structured findings
- BOM-managed dependencies are upgraded through the correct control point
- unsafe direct version insertion is blocked by validation
- approved fixes are committed to a new branch with a clear PR trail